Loading and Requiring

Speaking of *Namespaces (23)*, there is one interesting detail that involves Namespaces, constants, and Ruby's load and require methods. Imagine finding a motd.rb file on the web that displays a "message of the day" on the console. You want to add this code to your latest program, so you load the file to execute it and display the message:

```
load('motd.rb')
```

Using load, however, has a side effect. The motd.rb file probably defines variables and classes. Although variables fall out of scope when the file has finished loading, constants don't. As a result, motd.rb can pollute your program with the names of its own constants—in particular, class names.

You can force motd.rb to keep its constants to itself by passing a second, optional argument to load:

```
load('motd.rb', true)
```

If you load a file this way, Ruby creates an anonymous module, uses that module as a Namespace to contain all the constants from motd.rb, and then destroys the module.

The require method is quite similar to load, but it's meant for a different purpose. You use load to execute code, and you use require to import libraries. That's why require has no second argument: those leftover class names are probably the reason why you imported the file in the first place. Also, that's why require tries only once to load each file, while load executes the file again every time you call it.

Quiz: Missing Lines

Where you find your way around the Ruby object model.

Back in *The Truth About Classes, on page 19*, Bill showed you how objects and classes are related. As an example, he used a snippet of code and this whiteboard diagram:

